Pareto Core: The Critical 20% of SRE Mastery

Expert Panel Consensus

Staff/Principal Product SRE: The following items represent the minimum viable mental model for service ownership. Master these, and you can own a production service. Skip any, and you're a liability on-call.

SRE Hiring Manager: These concepts appear in 80%+ of our interview failures. Candidates who demonstrate practical competence here advance. Those who can only parrot definitions don't.

Curriculum Designer: This is designed for maximum ROI. Each item unlocks multiple debugging capabilities and interview signals. The remaining 80% of SRE knowledge can be learned on-the-job after these foundations.

---

1. The Four Golden Signals (Observability Foundation)

Why Hiring Managers Care: This is the universal language of service health. If you can't articulate these for your service, you don't understand your service.

The Signals

1. **Latency**: How long requests take (distribution matters more than average)
2. **Traffic**: Request volume and patterns
3. **Errors**: Rate of failed requests (explicit and implicit)
4. **Saturation**: Resource utilization approaching limits

Observable System Behavior

```
# High latency but low CPU
- Symptom: p99 latency = 5s, CPU = 20%
- Reality: Blocked on downstream service, database locks, or network
```

```
- Wrong diagnosis: "Need more CPU"

# Low error rate but users complaining
- Symptom: HTTP 200 responses, error rate = 0%
- Reality: Returning cached stale data, silent data corruption
- Wrong diagnosis: "Service is healthy"

# Low traffic but high saturation
- Symptom: 100 req/s, memory at 95%
- Reality: Memory leak, connection pool exhaustion
- Wrong diagnosis: "We can handle more load"
```

Production Grounding

Failure it Explains: August 2023, Payment Service outage. Error rate showed 0% because failing requests timed out before being counted. Latency metrics caught it (p99 went from 200ms to 30s).

Interview Signal: "Walk me through how you'd diagnose slow checkout" - candidates who jump to CPU/memory without checking downstream latency fail.

Lab Practice

Easy: Deploy microservice with Prometheus, add RED (Rate/Errors/Duration) metrics, identify which signal detects injected database slowness

Moderate: Multi-service setup where service A calls B calls C. Inject latency in C, prove it propagates, measure at each hop

Hard: Silent corruption scenario - service returns 200 OK but wrong data. Design metrics to detect this when error rate is 0%

References

- Google SRE Book: Chapter 6 "Monitoring Distributed Systems"
- Prometheus: histogram_quantile() for latency percentiles
- USE Method (Brendan Gregg): Utilization, Saturation, Errors for resources

Why the Remaining 80% Can Be Deferred

You don't need to know vanity metrics, custom dashboard artistry, or log aggregation patterns to start debugging. Master these four, and you can triage 80% of incidents.

2. SLIs, SLOs, and Error Budgets (Reliability Contracts)

Why Hiring Managers Care: This is what separates SRE from ops. If you can't define and negotiate these, you're not doing SRE.

Core Concepts

SLI (Service Level Indicator): Quantified measure of service behavior
- Good: "Fraction of HTTP requests completing < 500ms"
- Bad: "System is fast"

SLO (Service Level Objective): Target threshold for SLI
- Good: "99.9% of requests complete < 500ms over 30-day window"
- Bad: "We want 5 nines"

Error Budget: (1 - SLO) = allowed failure
- 99.9% SLO = 0.1% error budget = 43.2 minutes/month downtime
- When exhausted, velocity stops until reliability improves

Observable System Behavior

```
# Error budget exhausted
- Q1: 99.85% availability (target: 99.9%)
- Budget used: 150% (6.5 hours down vs 4.3 hour allowance)
- Action: Freeze non-critical launches, mandate reliability work

# False SLO compliance
- SLO: 99.9% of API requests succeed
- Reality: 0.5% error rate from edge cases not in SLO definition
- Users angry, but "we hit our SLO"
- Problem: SLI doesn't capture user experience
```

Production Grounding

Failure it Explains: Q4 2023, Auth Service launched risky feature when error budget was exhausted. Feature caused cascade failure, took down 5 services. Post-mortem: "Error budget policy violated."

Interview Signal: "How do you balance velocity and stability?" - wrong answer is "compromise"; right answer is "error budget mechanism."

Lab Practice

Easy: Define SLI/SLO for simple HTTP service. Calculate error budget. Simulate failures until budget exhausted.

Moderate: Multi-dependency service. Design SLI that captures both service errors AND downstream failures. Calculate composite SLO.

Hard: Product team wants to launch but error budget is 80% consumed with 10 days left in period. Model the risk: what's probability of SLO violation? What conditions would you require to approve launch?

References

- Google SRE Book: Chapter 4 "Service Level Objectives"
- SLO calculator: (1 - uptime_target) × seconds_in_period = allowed_downtime
- Error Budget Policy: explicit rules for when budget is exhausted

Why the Remaining 80% Can Be Deferred

You don't need SLA negotiations, multi-region SLO math, or user-journey SLIs on day one. Start with availability and latency for one service.

---

3. Incident Response Fundamentals (The First Hour)

Why Hiring Managers Care: Competence here determines MTTR. Weak incident response turns 5-minute outages into 3-hour disasters.

The Incident Response Lifecycle

1. **Detection** → Alert fires
2. **Triage** → Assess severity, page appropriate responders
3. **Mitigation** → Stop the bleeding (rollback, disable feature, scale up)
4. **Remediation** → Fix root cause
5. **Post-Mortem** → Learn and prevent recurrence

Critical Distinction: Mitigation ≠ Remediation
- Mitigation: Fast, tactical, often ugly
- Remediation: Slow, strategic, proper fix

Observable System Behavior

```
# Premature remediation (antipattern)
- Incident: Database connection pool exhausted
- Wrong: "Let's increase pool size" (takes 20 min to deploy)
- Right: "Restart service to reset pool" (mitigated in 2 min)
- Then: Investigate why pool exhausted (remediation)

# Severity misclassification
- Symptom: Payment API returns 500s
- Declared: SEV3 (low urgency)
- Reality: Revenue stopped, should be SEV1
- Result: 45-min delay before proper escalation

# Cascade during mitigation
- Incident: Service A overloaded
- Mitigation: Scaled A from 10 → 100 instances
- Result: Service A's increased traffic overloaded downstream service B
- New incident: Now B is down too
```

Production Grounding

Failure it Explains: June 2024, Search Service outage. Engineer spent 1 hour debugging root cause while users were down. Should have rolled back in 5 minutes, then debugged.

Interview Signal: "Service is returning 500s. What do you do in the first 5 minutes?" - candidates who start debugging fail; right answer is "check if recent deploy, rollback if yes."

Lab Practice

Easy: Service starts returning errors. You have rollback and scale-up options. Time yourself: detection to mitigation. Target < 5 min.

Moderate: Three-service dependency chain. Inject failure in middle service. Practice: escalate correctly, coordinate mitigation across teams (simulated), write incident update for stakeholders.

Hard: Database replica lag causes stale reads. Some users see outdated data. No errors logged. How do you detect? How do you mitigate without full outage? Simulate this.

References

- Google SRE Book: Chapter 14 "Managing Incidents"
- PagerDuty Incident Response Guide

- Severity matrix: User impact × business impact

Why the Remaining 80% Can Be Deferred

You don't need war room coordination protocols, multi-region failover orchestration, or executive communication templates to start. Learn detect-mitigate-remediate for single services first.

---

4. Distributed Systems Failure Modes (What Actually Breaks)

Why Hiring Managers Care: If you don't understand failure modes, you can't design resilient systems or debug outages.

The Critical Five

A. Partial Failures

Reality: In distributed systems, "up" and "down" are fuzzy. Parts fail independently.

```
# Classic partial failure
- Symptom: Service A can reach 9/10 instances of Service B
- Result: 10% of requests fail, but "Service B is up"
- Wrong fix: "B is healthy, A must have bugs"
- Right understanding: Network partition, degraded health, or flapping instance
```

B. Cascading Failures

Reality: One service's problem becomes everyone's problem.

```
# Timeout cascade
- Service B latency: 1s → 5s (gradual degradation)
- Service A default timeout: 10s
- Result: A's connection pool exhausts waiting for B
- Now A is also down, propagates upstream to C, D, E...
- Mitigation: Aggressive timeouts + circuit breakers
```

C. Thundering Herd

Reality: Many clients retry at once, amplifying failure.

```
# Cache invalidation stampede
- Redis cache layer goes down
- 10,000 req/s suddenly hit database directly
- Database overloads, starts timing out
- Timeouts trigger retries → 20,000 req/s → death spiral
- Mitigation: Exponential backoff, jitter, request coalescing
```

D. Split Brain

Reality: Two parts of system think they're authoritative.

```
# Leadership split brain
- Network partition isolates leader election nodes
- Both sides elect new leaders
- Both leaders accept writes
- Conflict detected hours later
- Result: Data corruption, requires manual reconciliation
```

E. Resource Exhaustion (Non-CPU)

Reality: Services fail from non-obvious resource limits.

```
# Connection pool exhaustion
- CPU: 20%, Memory: 40%
- But: All 50 DB connections in use
- New requests queue, timeout, error
- Metric that matters: connection pool utilization, not CPU

# File descriptor exhaustion
- Linux default: 1024 file descriptors per process
- Each HTTP request = 1 FD
- Under load: "Too many open files" errors
- Not in standard golden signals
```

Production Grounding

Failure it Explains: March 2024, Notification Service outage. Service scaled to 100 instances, exhausted connection pool on shared database. Should have implemented connection limits per instance.

Interview Signal: "Design a rate limiter. What failure modes do you need to consider?" - candidates who only think about happy path fail.

Lab Practice

Easy: Two services with 10s timeout. Inject 15s latency in downstream. Observe connection pool exhaustion. Fix with circuit breaker.

Moderate: Cache-aside pattern. Kill cache. Measure thundering herd on database. Implement request coalescing or probabilistic early recomputation.

Hard: Three-service mesh. Inject partial network partition (80% packet loss between specific pairs). Watch cascade. Measure blast radius. Design bulkheads to contain failure.

References

- Google SRE Book: Chapter 22 "Addressing Cascading Failures"
- "Designing Data-Intensive Applications" (Kleppmann): Chapter 8
- Hystrix: Circuit breaker library documentation

Why the Remaining 80% Can Be Deferred

You don't need consensus algorithms, distributed transactions, or vector clocks to start. These five modes explain 80% of prod outages.

---

5. Prometheus + Grafana (Observability Stack)

Why Hiring Managers Care: This is the industry standard. If you can't set this up and query it effectively, you're not interview-ready.

Why Prometheus

Pull-based: Prometheus scrapes metrics from targets. Allows:
- Service discovery (don't manually configure every instance)

- Targets can be ephemeral (containers come and go)
- Prometheus determines scrape interval

Time-series database: Stores metrics as (timestamp, value) pairs with labels

PromQL: Query language for aggregation, rates, percentiles

Essential PromQL Patterns

```
# Request rate (requests per second)
rate(http_requests_total[5m])

# Error rate (percentage)
rate(http_requests_total{status=~"5.."}[5m])
  /
rate(http_requests_total[5m])

# Latency percentiles
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))

# Saturation (resource utilization)
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes
```

Observable System Behavior

```
# Metric cardinality explosion
- Bad: user_id label on every metric
- Result: Millions of time series, Prometheus OOM
- Right: Use user_id in logs, not metrics

# Rate vs increase confusion
- rate(): per-second rate over time window
- increase(): absolute increase over time window
- Common error: Using increase() for alerting (wrong for varying intervals)

# Missing scrape targets
- Service deployed, but not in Prometheus targets
- No metrics appear, looks like service is down
- Reality: Prometheus can't find it (service discovery config wrong)
```

Production Grounding

Failure it Explains: April 2024, dashboards showed "no data" during incident. Engineers flew blind. Problem: Prometheus server itself was down. Needed HA Prometheus setup.

Interview Signal: "How would you alert on high error rate?" - candidates who suggest "check logs" fail; right answer involves PromQL alert rule.

Lab Practice

Easy: Deploy Prometheus + Grafana via Docker Compose. Scrape metrics from sample app. Create dashboard with RED metrics. Write alert rule for error rate > 5%.

Moderate: Multi-service setup. Implement histogram metrics for latency. Create dashboard showing p50/p95/p99. Intentionally create cardinality explosion, observe Prometheus memory, fix it.

Hard: Simulate Prometheus server failure during incident. Implement HA Prometheus (two Prometheus servers scraping same targets). Configure Grafana to query both. Verify no gap in metrics during single-server failure.

References

- Prometheus Documentation: Best Practices
- Grafana Dashboard: "USE Method" and "RED Method" templates
- Blog: "Prometheus Histograms vs Summaries"

Why the Remaining 80% Can Be Deferred

You don't need Thanos for long-term storage, VictoriaMetrics for scale, or custom exporters on day one. Master Prometheus + Grafana basics first.

---

6. Retry Logic & Circuit Breakers (Resilience Patterns)

Why Hiring Managers Care: Every production service needs these. If you don't implement them, you're building fragile systems.

Retry Logic (Done Right)

Core Pattern: Exponential backoff + jitter

```
# Bad retry (thundering herd)
for attempt in range(3):
    try:
        return call_api()
    except:
        time.sleep(1)  # All clients retry at same time

# Good retry
import random

def retry_with_backoff(func, max_attempts=3):
    for attempt in range(max_attempts):
        try:
            return func()
        except RetryableError as e:
            if attempt == max_attempts - 1:
                raise
            delay = (2 ** attempt) + random.uniform(0, 1)
            time.sleep(delay)
```

Critical Constraints:
- Only retry idempotent operations
- Only retry transient failures (timeout, 5xx), not permanent (4xx)
- Set max retries (don't retry forever)
- Add jitter to prevent thundering herd

Circuit Breaker

States: Closed → Open → Half-Open → Closed

```
Closed (Normal):
- Requests pass through
- Track failure rate
- If failure rate > threshold → Open

Open (Failing):
- Requests fail fast (don't attempt)
- After timeout → Half-Open

Half-Open (Testing):
- Allow limited requests through
```

```
- If succeed → Closed
- If fail → Open
```

Observable System Behavior

```
# Retry storm amplification
- Service B has 1% error rate
- Service A retries 3x on error
- Effective error rate: 1% * 3 = 3% load increase
- If 10 services do this: 30% more load from retries alone
- Under stress, this causes cascade

# Circuit breaker false positive
- Service B has 10s P99 latency spike
- A's circuit breaker threshold: 5s timeout
- Circuit opens, A stops calling B
- B recovers in 30s
- Circuit stays open for 60s (timeout period)
- Result: Unnecessary 30s of failures

# Missing idempotency causes duplicate charges
- Payment API call times out
- Client retries (doesn't know if first succeeded)
- First call actually succeeded
- Result: Customer charged twice
- Fix: Idempotency keys
```

Production Grounding

Failure it Explains: July 2024, Authentication service blip (30s downtime). Every downstream service retried aggressively, amplified load 5x, prevented Auth from recovering. Should have had circuit breakers.

Interview Signal: "How do you handle calling an unreliable dependency?" - candidates who say "just retry" fail; need exponential backoff, circuit breaker, and discussion of idempotency.

Lab Practice

Easy: Service A calls B. Inject 50% failure rate in B. Implement naive retry, measure amplification factor. Fix with exponential backoff + jitter.

Moderate: Implement circuit breaker. Inject failures in B. Watch circuit open. Verify fail-fast behavior. Watch half-open probing. Tune thresholds.

Hard: Multi-service chain (A→B→C). Each has retries. Inject 5% error rate in C. Measure amplification at each hop. Prove retry storm. Fix with circuit breakers + retry budgets.

References

- Google SRE Book: Chapter 22 "Addressing Cascading Failures"
- Netflix Hystrix: Circuit breaker implementation
- "Release It!" (Nygard): Stability Patterns

Why the Remaining 80% Can Be Deferred

You don't need bulkheads, adaptive timeouts, or hedge requests on day one. Master retries + circuit breakers first.

---

7. Structured Logging & Distributed Tracing (Debugging at Scale)

Why Hiring Managers Care: Logs are your incident investigation tool. If you can't correlate logs across services, you can't debug distributed systems.

Structured Logging

Bad (Unstructured):

```
log.info(f"User {user_id} requested {item} at {timestamp}")
```

- Can't query by user_id without regex
- Can't aggregate by item
- Parsing is brittle

Good (Structured):

```
{
  "timestamp": "2024-02-06T14:32:11Z",
  "level": "info",
```

```
  "message": "item_requested",
  "user_id": "user123",
  "item_id": "item456",
  "request_id": "req-789",
  "trace_id": "trace-abc"
}
```

- Queryable: `WHERE user_id = 'user123'`
- Aggregatable: `GROUP BY item_id`
- Traceable across services via trace_id

Distributed Tracing

Problem: Single user request touches 10 services. How do you see the full path?

Solution: Trace ID propagated through headers

```
User → API Gateway → Auth → Product → Payment
       [trace-abc]   [trace-abc] [trace-abc]  [trace-abc]

Query: "Show me all logs for trace-abc"
Result: See full request path with timing
```

OpenTelemetry: Standard for propagating trace context

Observable System Behavior

```
# Missing trace context
- Incident: Payment failures
- Logs show: "Payment failed" in Payment service
- But: Can't find what called Payment service
- Can't correlate with user action
- Root cause investigation takes 2 hours instead of 5 minutes

# Log volume explosion
- Service logs every HTTP request
- Under normal load: 1 GB/day
- Under attack: 100 GB/day
- Cost spike: $3000/month extra
- Fix: Sample logs (log 1% of requests, 100% of errors)

# Logs without context
```

```
{
  "message": "Database connection failed"
}
# Missing: which database? which query? which user affected?
# Result: Can't reproduce or debug
```

Production Grounding

Failure it Explains: September 2024, intermittent checkout failures. Couldn't correlate failed checkouts across 5 services. Took 6 hours to find root cause. After implementing tracing: same class of issue debugged in 15 minutes.

Interview Signal: "How do you debug a request that fails only for some users?" - candidates who suggest "check logs" without mentioning correlation fail.

Lab Practice

Easy: Add structured logging to service. Emit JSON logs. Query logs in Loki/ElasticSearch by specific fields.

Moderate: Multi-service setup. Generate unique trace_id at API gateway. Propagate via headers. Correlate logs across services for single request.

Hard: Implement sampling (log 1% of successful requests, 100% of errors). Prove log volume reduced. Intentionally create error, verify it's logged despite sampling.

References

- Google SRE Book: Chapter 18 "Software Engineering in SRE"
- OpenTelemetry: Trace context propagation
- W3C Trace Context specification

Why the Remaining 80% Can Be Deferred

You don't need span tags, baggage propagation, or trace sampling strategies on day one. Master trace ID correlation first.

---

8. Capacity Planning Fundamentals (Headroom & Saturation)

Why Hiring Managers Care: Services fail when they run out of capacity. Knowing when to scale is non-negotiable.

Key Concepts

Utilization: Percentage of resource in use
- Example: CPU at 70%

Saturation: Resource queueing or unable to meet demand
- Example: Thread pool with queue depth = 50

Critical Insight: High utilization ≠ saturation

```
# CPU Utilization: 80%
# Saturation: 0 (no queuing)
# Status: Healthy

# CPU Utilization: 40%
# Saturation: High (requests queuing)
# Status: Unhealthy (bottleneck elsewhere)
```

Headroom Calculation

Formula: (Max capacity - Current usage) / Growth rate = Time until exhaustion

```
Example:
- Current traffic: 1000 req/s
- Max capacity (before degradation): 1500 req/s
- Headroom: 500 req/s
- Growth rate: 50 req/s per week
- Time to exhaustion: 500/50 = 10 weeks

Action: Add capacity before 10 weeks
```

Observable System Behavior

```
# False headroom (wrong capacity metric)
- CPU: 30% (looks safe)
- But: Connection pool at 95% utilization
- Next traffic spike: Connection exhaustion, errors
- Wrong metric: CPU
- Right metric: Connection pool saturation

# Non-linear degradation
- Load test: 1000 req/s → p99 = 100ms (great!)
- Production: 1100 req/s → p99 = 5000ms (disaster!)
- Reason: Above certain threshold, queue depth explodes
- Capacity planning must account for non-linear cliffs

# Uneven shard distribution
- 10 shards, even data distribution
- Shard 1: 1000 req/s
- Shard 7: 5000 req/s (hot shard)
- Average utilization: 30%
- Shard 7 utilization: 90% (saturated)
- Can't scale based on average
```

Production Grounding

Failure it Explains: Black Friday 2023, e-commerce site down. Capacity planning based on CPU. Actual bottleneck: Database connection pool. Should have load tested specific resource limits.

Interview Signal: "How do you know when to scale your service?" - candidates who say "when CPU is high" fail; need discussion of multiple resource types and saturation.

Lab Practice

Easy: Load test service to find capacity limit. Plot latency vs load. Identify non-linear degradation point.

Moderate: Service with database connection pool (size=10). Under load, find the request rate where pool exhaustion begins. Calculate headroom based on growth rate.

Hard: Multi-shard system with uneven distribution. Monitor per-shard saturation. Implement autoscaling that responds to highest-saturation shard, not average.

References

- Google SRE Book: Chapter 27 "Reliable Product Launches at Scale"
- USE Method (Brendan Gregg): For every resource, check Utilization, Saturation, Errors
- Queueing theory: Little's Law

Why the Remaining 80% Can Be Deferred

You don't need demand forecasting models, multi-region capacity management, or over-provisioning strategies on day one. Master utilization vs saturation first.

---

9. Post-Mortem Practice (Learning from Failure)

Why Hiring Managers Care: Post-mortems are how SRE teams improve. If you can't write a good post-mortem, you can't drive reliability improvements.

Blameless Post-Mortem Structure

1. **Summary**: What happened (1 paragraph)
2. **Timeline**: Chronological events with timestamps
3. **Root Cause(s)**: What actually failed (technical)
4. **Contributing Factors**: What made it worse
5. **Impact**: User/business effect (quantified)
6. **Action Items**: Specific, assigned, tracked

Blameless Principle: Focus on system failures, not human error

Observable System Behavior

```
# Blame-focused (bad)
"Engineer deployed buggy code without testing"
→ Action: "Engineers must test better"
→ Result: No system improvement

# System-focused (good)
"Deployment lacked automated integration tests; load test environment
differed from production; deploy proceeded despite test failure"
→ Actions:
```

```
  1. Add integration tests to CI/CD
  2. Synchronize test/prod environments
  3. Implement deploy gates (tests must pass)
→ Result: Systemic improvement

# Vague action items (bad)
- "Improve monitoring"
- "Test more thoroughly"
- "Communicate better"

# Specific action items (good)
- "Add p99 latency alert for /checkout endpoint (owner: Alice, due: Feb 15)"
- "Implement connection pool exhaustion metric (owner: Bob, due: Feb 20)"
- "Create runbook for database failover (owner: Charlie, due: Feb 25)"
```

Production Grounding

Failure it Explains: After 2022 outage, post-mortem said "improve testing." No specific actions. Same class of failure recurred in 2023. After 2023 outage, specific actions taken (CI/CD gates added). No recurrence since.

Interview Signal: "Tell me about an outage you caused." - candidates who blame themselves or others fail; good candidates explain system gaps and improvements.

Lab Practice

Easy: Simulate incident (service deploy causes errors). Write post-mortem following template. Include timeline, root cause, action items.

Moderate: Multi-service incident with unclear root cause. Investigate using logs/metrics. Write post-mortem explaining how you determined causality.

Hard: Incident with multiple contributing factors (deploy + traffic spike + dependency failure). Write post-mortem balancing immediate vs systemic causes. Prioritize action items by impact.

References

- Google SRE Book: Chapter 15 "Postmortem Culture: Learning from Failure"
- Etsy: "Debriefing Facilitation Guide"
- Example post-mortems: GitHub incident reports (public)

Why the Remaining 80% Can Be Deferred

You don't need facilitation training, incident review boards, or post-mortem databases on day one. Master the template and blameless mindset first.

---

10. On-Call Fundamentals (Sustainable Operations)

Why Hiring Managers Care: On-call quality determines retention. Bad on-call = engineers quit. Good on-call = sustainable operations.

Alert Design Principles

Symptom-based, not cause-based

```
# Bad (cause-based)
Alert: "Redis is down"
Problem: Redis might be down but app is fine (caching degraded gracefully)
Result: False page at 3 AM

# Good (symptom-based)
Alert: "Error rate > 5% for 5 minutes"
Problem: Alerts only when users are impacted
Result: Actionable page
```

Actionable, not informational

```
# Bad
Alert: "Disk space at 80%"
Action: ???

# Good
Alert: "Disk space at 80%, projected to fill in 4 hours"
Action: Clear logs, provision more disk, or scale up
```

Alert Fatigue

Reality: Too many alerts → ignored alerts → missed real incidents

```
# Statistics from real SRE teams
- Average on-call week: 50-100 pages
- Actionable pages: 10-20
- False positives: 40-80
- Result: Engineers ignore pages, miss real outages
```

Solution: Alert tuning

1. Fix false positives immediately
2. Group related alerts (one page for multi-symptom incident)
3. Use low/medium/high severity (page only for high)
4. Review alert effectiveness monthly

Observable System Behavior

```
# Alert storm
- Database latency increases
- 50 alerts fire simultaneously:
  - "API latency high"
  - "Queue depth high"
  - "Error rate elevated"
  - "Database connection pool high"
- All are symptoms of same root cause
- On-call engineer overwhelmed
- Fix: Group alerts, page once with "Service degraded"

# Missing alert
- Incident: 30-minute outage
- No alert fired
- Root cause: Alert threshold too high (error rate > 50%, actual rate was 30%
- Fix: Tune threshold based on actual SLO

# Flapping alert
- Alert fires → resolves → fires → resolves (every 2 minutes)
- Pages on-call engineer 20 times
- Reality: Metric crosses threshold repeatedly (needs hysteresis)
- Fix: Require problem to persist for 5 minutes before alerting
```

Production Grounding

Failure it Explains: Q3 2023, on-call rotation had 80% attrition. Exit interviews: "Too many false alerts, couldn't sleep." After alert tuning: 70% reduction in pages, attrition dropped to 10%.

Interview Signal: "How do you decide what to alert on?" - candidates who say "alert on everything" fail; need discussion of symptom-based alerts and alert fatigue.

Lab Practice

Easy: Service with 5 alerts. Intentionally trigger each. Determine which are actionable. Disable or tune non-actionable ones.

Moderate: Simulate incident where root cause triggers 10 related alerts. Design alert grouping strategy. Test that one page is sent instead of 10.

Hard: Week-long alert analysis. Given alert logs, calculate: false positive rate, mean time between alerts, alert action taken rate. Propose tuning strategy to reduce pages by 50% without missing real incidents.

References

- Google SRE Book: Chapter 6 "Monitoring Distributed Systems" (pages vs tickets)
- PagerDuty: "Alert Fatigue" guide
- "My Philosophy on Alerting" (Rob Ewaschuk, Google)

Why the Remaining 80% Can Be Deferred

You don't need on-call handoff protocols, escalation policies, or follow-the-sun rotations on day one. Master alert design and fatigue reduction first.

Justification: Why These 10 Cover 80%

Incident Commander Perspective: In 100+ incidents, these 10 concepts appeared in 90+. Observability (golden signals, Prometheus), failure modes, and incident response are the foundation. Everything else is learned on-the-job.

SRE Hiring Manager Perspective: Phone screens filter on 5 of these (golden signals, SLOs, retry/circuit breaker, post-mortems, alert design). On-site interviews cover distributed systems and capacity planning. Master these, and you pass.

Curriculum Designer Perspective: These 10 can be learned in 90 days with focused practice. They unlock debugging capabilities immediately. The remaining 80% (consensus algorithms, multi-region failover, chaos engineering at scale) requires production experience to appreciate.

What's Deferred (The Remaining 80%):
- Advanced observability (tracing analysis, custom exporters, metric aggregation)
- Platform SRE (Kubernetes operations, network policies, storage classes)
- Release engineering (canary deploys, blue/green, feature flags)
- Chaos engineering (failure injection frameworks, game days)
- Multi-region reliability (global load balancing, data replication)
- Cost optimization (resource right-sizing, spot instances)
- Security SRE (threat modeling, secrets management)
- Organizational SRE (SRE engagement models, embedded vs centralized)

These are important but don't block basic competence. Learn the Pareto Core first.

Self-Check Before Proceeding

Panel Question: Would a hiring manager trust a candidate who mastered these 10 over a bootcamp graduate?

Staff SRE: Yes. These are the concepts I check for in interviews.

Incident Commander: Yes. This is the minimum competence for on-call.

Hiring Manager: Yes. This demonstrates production thinking, not just tool familiarity.

Verdict: Proceed to detailed explanations and labs.